

Rendu individuel

Adame Nianghane
Observabilité & sécurité

Auteur	Adame Nianghane
Rôle	Observabilité & sécurité
Classe / Promo	M2 DO C — 2025-2026
Période	Janvier – Juin 2026
Projet	Plateforme de gestion d'un cabinet d'audioprothèse
Domaine	Supervision (Prometheus/Grafana/Loki), DevSecOps, durcissement

SUP DE VINCI — Expert en Systèmes d'Information — Mastère DevOps — Classe **M2 DO C** — Promotion 2025-2026. Document réalisé dans le cadre du projet d'étude de fin d'année.

1. Contexte et rôle dans le projet

J'ai pris en charge deux volets complémentaires qui déterminent la confiance que l'on peut accorder à un système en production : l'**observabilité**, c'est-à-dire la capacité à comprendre ce que fait la plateforme à tout instant, et la **sécurité**, tant celle de la chaîne logicielle que celle de l'exécution. Ces deux sujets partagent une même philosophie : anticiper les problèmes plutôt que les subir, en rendant le système transparent et en le durcissant à chaque étape.

Ce positionnement m'a amené à travailler sur l'ensemble du périmètre : j'ai instrumenté l'application développée par Zaafir, intégré mes analyses dans la chaîne d'Elyess, et sécurisé les objets Kubernetes packagés par Anis. La sécurité et l'observabilité sont en effet des préoccupations transverses, qui ne peuvent être traitées en silo.

2. Ma contribution détaillée

2.1 Collecte des métriques (Prometheus)

J'ai déployé Prometheus, qui collecte périodiquement les métriques exposées par les différents composants. L'application publie ses propres indicateurs — nombre de requêtes, latence, code de réponse — que j'ai fait découvrir automatiquement par Prometheus grâce à un objet de configuration dédié. Ainsi, dès qu'une nouvelle instance de l'application démarre, elle est automatiquement prise en compte par la supervision, sans configuration manuelle.

J'ai veillé à limiter la cardinalité des métriques, c'est-à-dire à éviter une explosion du nombre de séries temporelles, en agrégeant les mesures par route plutôt que par requête individuelle. Ce souci rejoint la démarche FinOps de l'équipe, car une supervision mal maîtrisée peut consommer beaucoup de mémoire et de stockage.

2.2 Visualisation (Grafana) et tableaux de bord

J'ai déployé Grafana et conçu un tableau de bord dédié à l'application, présentant les indicateurs clés : le débit de requêtes par route, la latence au 95e centile, le taux d'erreurs serveur et la disponibilité du backend. Ces quatre indicateurs correspondent aux « signaux dorés » de la supervision d'un service, et permettent de diagnostiquer rapidement une dégradation.

J'ai automatisé l'import de ce tableau de bord afin qu'il soit présent dès l'installation, sans manipulation. J'ai également exposé Grafana de façon accessible via l'Ingress existant, en réutilisant l'adresse publique du cluster — un choix qui évite de provisionner une seconde adresse IP et participe donc à la maîtrise des coûts.

2.3 Centralisation des journaux (Loki)

Pour les journaux, j'ai déployé Loki accompagné de son agent de collecte, qui récupère les traces de tous les composants et les rend consultables et corrélables depuis Grafana. L'application produit des journaux structurés, ce qui facilite leur exploitation. Le choix de Loki plutôt qu'une pile Elasticsearch a été motivé, en accord avec l'équipe, par son empreinte réduite : Loki n'indexe que les étiquettes, ce qui le rend bien plus économe pour un usage équivalent.

2.4 Alerting (Alertmanager)

J'ai défini des règles d'alerte qui se déclenchent lorsque le service devient indisponible, lorsque le taux d'erreurs dépasse un seuil ou lorsque la latence se dégrade. Ces alertes sont routées par Alertmanager vers une messagerie d'équipe (Slack ou Teams) au moyen d'un point d'entrée dédié. L'objectif est de prévenir l'équipe d'un incident avant même que les utilisateurs ne s'en aperçoivent.

2.5 Sécurité de la chaîne (DevSecOps)

J'ai intégré dans la chaîne d'intégration trois analyses complémentaires. Trivy examine les dépendances, les fichiers d'infrastructure et les images à la recherche de vulnérabilités connues, et bloque le déploiement en cas de faille critique. CodeQL réalise une analyse statique du code source pour détecter des schémas dangereux. Gitleaks recherche toute fuite de secret dans l'historique. Les

résultats sont publiés dans l'onglet de sécurité du dépôt, au format standard SARIF.

2.6 Durcissement de l'exécution

Au-delà des analyses, j'ai durci l'exécution : les conteneurs s'exécutent sans privilèges et sans possibilité d'élévation, les capacités système superflues sont retirées, et l'accès à l'API du cluster est régi par des rôles restreints (RBAC). Les communications avec la base sont chiffrées en TLS, et les secrets ne sont jamais écrits dans le dépôt : ils transitent chiffrés et sont injectés au dernier moment. J'ai également préparé des politiques de cloisonnement réseau limitant les communications entre composants au strict nécessaire.

3. Choix techniques et justifications

Le triptyque Prometheus / Grafana / Loki s'est imposé comme la référence open source de l'observabilité cloud-native, avec une intégration étroite entre métriques et journaux au sein d'une interface unique. Pour la sécurité, j'ai retenu des outils gratuits et largement adoptés, complémentaires dans leur périmètre (dépendances, code, secrets), afin de couvrir la chaîne sans coût de licence.

J'ai fait le choix d'intégrer ces analyses directement dans la chaîne d'intégration, plutôt que de les exécuter ponctuellement : la sécurité devient ainsi une propriété vérifiée à chaque livraison, selon le principe du « décalage vers la gauche » (shift-left) qui consiste à détecter les problèmes au plus tôt.

4. Difficultés rencontrées et solutions apportées

La principale difficulté a été de faire tenir une pile de supervision complète sur un cluster volontairement réduit pour des raisons budgétaires. J'ai dû dimensionner soigneusement les ressources allouées à chaque composant et accepter des durées de rétention courtes, afin que la supervision n'entre pas en concurrence avec l'application pour les ressources du cluster.

Une autre difficulté a concerné la découverte automatique des métriques de l'application : il a fallu s'assurer que Prometheus sélectionne bien l'objet de configuration produit par le déploiement, ce qui supposait un bon ordonnancement de l'installation (les définitions de ressources devant précéder l'application). Ce point a été résolu en coordination avec le responsable de la chaîne d'intégration.

5. Perspectives d'évolution

Je souhaiterais ajouter le **traçage distribué**, au moyen d'OpenTelemetry et d'un outil comme Tempo ou Jaeger, afin de suivre une requête à travers les composants et de diagnostiquer finement les lenteurs — un troisième pilier de l'observabilité, complémentaire des métriques et des journaux.

Je centraliserais également la gestion des secrets dans un coffre dédié offrant rotation automatique, et je renforcerais la posture de sécurité du cluster par des politiques avancées : standards de sécurité des pods, contrôleur d'admission validant les manifestes, et signature des images pour garantir leur provenance. Enfin, je définirais des objectifs de niveau de service (SLO) assortis d'un suivi de leur consommation.

6. Analyse critique des limites

Le routage des alertes vers une messagerie nécessite un point d'entrée externe qui n'a pas été connecté en production dans le cadre du projet ; le mécanisme est en place mais sa validation de bout en bout reste à finaliser. De même, les politiques de cloisonnement réseau que j'ai préparées supposent un module réseau capable de les appliquer, que nous n'avons pas activé sur le cluster d'étude pour rester légers : elles sont donc fournies mais non strictement appliquées.

Par ailleurs, une véritable conformité à l'hébergement de données de santé imposerait une région et des services certifiés que nous n'avons pas retenus dans ce cadre, et la rétention volontairement courte des métriques et journaux limite l'analyse rétrospective des incidents anciens.

7. Annexe — documentation utilisateur (mon périmètre)

7.1 Consulter la supervision

L'utilisateur accède à Grafana via son adresse dédiée, avec les identifiants fournis. Le tableau de bord « Vue d'ensemble » est disponible immédiatement et présente l'état de santé de l'application. Pour observer l'évolution des courbes, il suffit de générer un peu de trafic sur l'API.

7.2 Explorer les journaux

Depuis Grafana, la vue d'exploration permet d'interroger la source de données Loki et de filtrer les journaux par composant, ce qui est précieux pour diagnostiquer un comportement anormal.

7.3 Consulter les rapports de sécurité

Les résultats des analyses de sécurité sont consultables dans l'onglet « Security » du dépôt, où chaque vulnérabilité détectée est listée avec sa gravité et sa localisation, permettant un suivi et une priorisation des corrections.

8. Analyse personnelle

Défis rencontrés

Mon principal défi a été de concilier une observabilité riche et une sécurité exigeante avec la contrainte de sobriété du projet. Il m'a fallu arbitrer en permanence entre l'exhaustivité souhaitable et ce que le cluster pouvait raisonnablement supporter.

Forces

Je retiens mon sens du détail sur les questions de sécurité et de conformité, ainsi que ma capacité à rendre un système transparent et compréhensible pour l'ensemble de l'équipe, ce qui a facilité le diagnostic collectif des incidents.

Faiblesses

Je dois concrétiser jusqu'au bout certaines mises en œuvre que j'ai préparées mais pas totalement activées, comme le routage effectif des alertes et l'application stricte des politiques réseau. J'ai parfois privilégié la mise en place sur la validation finale.

Compétences développées

J'ai approfondi ma maîtrise de Prometheus, Grafana et Loki, ainsi que des outils DevSecOps (Trivy, CodeQL, Gitleaks) et du durcissement Kubernetes. J'ai surtout intégré la sécurité et l'observabilité comme des réflexes de conception, et non comme des ajouts de dernière minute.

Axes d'amélioration personnels

Je souhaite mettre en œuvre le traçage distribué et des objectifs de niveau de service, industrialiser la gestion des secrets et la signature des images, et automatiser la validation de bout en bout de la chaîne d'alerte afin de fournir une observabilité complète et éprouvée.